

Python Tuple

What is a Tuple?



- **A tuple is an ordered, immutable collection of elements in Python.**
- **Key Characteristics:**
 - Ordered: Elements maintain their position (index-based access)
 - Immutable: Cannot be modified after creation (no add, remove, or change)
 - Heterogeneous: Can store different data types together
 - Indexed: Each element has a unique index starting from 0
- **Tuples are written with parentheses () or even without them:**

```
my_tuple = (1, 2, 3)
my_tuple = 1, 2, 3    # Also valid!
```

Creating Tuples



- **Empty Tuple:**

```
empty = ()  
empty = tuple()
```

- **Single Element Tuple (Note the comma!):**

```
single = (5,)          # Correct - comma is mandatory  
not_a_tuple = (5)      # Wrong - this is just an integer
```

- **Multiple Elements:**

```
t1 = (1, 2, 3, 4)  
t2 = 'a', 'b', 'c'  
t3 = (1, 'hello', 3.14, True)
```

Creating Tuples



- **Nested Tuples:**

```
nested = (1, (2, 3), (4, 5, 6))
```

Tuple Packing & Unpacking



- **Packing: Assigning multiple values to a single variable**

```
t = 10, 20, 30      # Values are 'packed' into a tuple
```

- **Unpacking: Assigning tuple elements to multiple variables**

```
a, b, c = t          # a=10, b=20, c=30
```

Accessing Tuple Elements



- **Indexing (0-based):**

```
t = ('apple', 'banana', 'cherry', 'date')
```

```
t[0]    → 'apple'
```

```
t[1]    → 'banana'
```

```
t[-1]   → 'date'      (last element)
```

```
t[-2]   → 'cherry'    (second last)
```

- **Slicing:**

```
t[1:3]   → ('banana', 'cherry')
```

```
t[:2]    → ('apple', 'banana')
```

```
t[2:]    → ('cherry', 'date')
```

```
t[::2]   → ('apple', 'cherry')    (step=2)
```

```
t[::-1]  → ('date', 'cherry', 'banana', 'apple') (reversed)
```

Tuple Operators



- **Concatenation (+): Joins two tuples**

`(1, 2) + (3, 4) → (1, 2, 3, 4)`

- **Repetition (*): Repeats a tuple**

`('Hi',) * 3 → ('Hi', 'Hi', 'Hi')`

- **Membership (in / not in): Check existence**

`3 in (1, 2, 3) → True`

`5 not in (1, 2, 3) → True`

- **Comparison Operators:**

`(1, 2) == (1, 2) → True`

`(1, 2) < (1, 3) → True (lexicographic comparison)`

`(1, 2) > (1, 1) → True`

Built-in Functions (Part 1)



- **len()** → Returns the number of elements

`len((1, 2, 3))` → 3

- **max()** → Returns the maximum element

`max((10, 50, 20))` → 50

- **min()** → Returns the minimum element

`min((10, 50, 20))` → 10

- **sum()** → Returns the sum of all elements

`sum((1, 2, 3, 4))` → 10

Built-in Functions (Part 1)



- **sorted()** → Returns a sorted list (tuple remains unchanged)

`sorted((3, 1, 4, 1)) → [1, 1, 3, 4]`

`tuple(sorted((3, 1, 4))) → (1, 3, 4)`

- **tuple()** → Converts other iterables to tuple

`tuple([1, 2, 3]) → (1, 2, 3)`

`tuple('abc') → ('a', 'b', 'c')`

`tuple({1, 2, 3}) → (1, 2, 3)`

Built-in Functions (Part 2)



- **enumerate()** → Returns index-value pairs

```
list(enumerate(('a', 'b', 'c')))  
→ [(0, 'a'), (1, 'b'), (2, 'c')]
```

- **zip()** → Aggregates elements from multiple iterables

```
list(zip((1, 2, 3), ('a', 'b', 'c')))  
→ [(1, 'a'), (2, 'b'), (3, 'c')]
```

- **any()** / **all()** → Check truthiness of elements

```
any((0, 1, 0)) → True  
all((1, 2, 3)) → True
```

Tuple Methods



- **count(x)** → Returns the number of times x appears

```
t = (1, 2, 2, 3, 2, 4)
```

```
t.count(2) → 3
```

- **index(x)** → Returns the first index of x

```
t = ('apple', 'banana', 'cherry')
```

```
t.index('banana') → 1
```

- **index(x, start, end)** → Search within a range

```
t = (1, 2, 3, 2, 4, 2)
```

```
t.index(2, 2) → 3 (search from index 2)
```

```
t.index(2, 2, 5) → 3 (search from index 2 to 4)
```

Immutability Explained



- **Tuples are immutable — but what does that really mean?**

```
t = (1, [2, 3], 4)
```

```
t[0] = 10          #      TypeError: 'tuple' object does not  
                   #      support item assignment
```

```
t[1].append(5)     #      Works! The list inside is mutable  
# t is now (1, [2, 3, 5], 4)
```

- **Key Insight:**

Immutability means the tuple's REFERENCES cannot change.

But mutable objects inside (like lists) can still be modified!

- **Use tuples when you want to prevent accidental changes**
to the structure of your data.

Tuple vs List

Feature	Tuple	List
Syntax	(1, 2, 3)	[1, 2, 3]
Mutable?	No	Yes
Methods	2 (count, index)	11+ methods
Memory Usage	Less	More
Iteration Speed	Faster	Slower
Use as dict key	Yes	No
Use Case	Fixed data, records, coordinates	Dynamic data, collections that change

- **Rule of Thumb: Use tuples for data that should not change.**

Common Pitfalls to Avoid



- **Forgetting the comma in single-element tuples:**

```
t = (5)      # This is an integer, NOT a tuple!  
t = (5,)    # This is a tuple with one element ✓
```

- **Trying to modify a tuple directly:**

```
t = (1, 2, 3)  
t[0] = 10    # TypeError!
```

Summary



Operation	Example	Result
Create	<code>(1, 2, 3)</code>	<code>(1, 2, 3)</code>
Access	<code>t[0]</code>	First element
Slice	<code>t[1:3]</code>	Elements 1 to 2
Concatenate	<code>t1 + t2</code>	Combined tuple
Repeat	<code>t * 3</code>	Tuple repeated
Length	<code>len(t)</code>	Number of items
Count	<code>t.count(x)</code>	Occurrences of x
Index	<code>t.index(x)</code>	First index of x
Membership	<code>x in t</code>	True / False
Min/Max/Sum	<code>min(t), max(t), sum(t)</code>	Computed value
Convert to list	<code>list(t)</code>	List version
Convert from list	<code>tuple(lst)</code>	Tuple version
Unpacking	<code>a, b = t</code>	<code>a=t[0], b=t[1]</code>

Summary



- **Tuples are ordered, immutable collections in Python.**
- **Created with parentheses () or comma-separated values.**
- **Support indexing, slicing, concatenation, repetition, and membership.**
- **Two built-in methods: `count()` and `index()`.**
- **Built-in functions: `len()`, `max()`, `min()`, `sum()`, `sorted()`, `tuple()`, etc.**
- **Tuple unpacking is a powerful and Pythonic feature.**
- **Use tuples for fixed data, records, coordinates, and dictionary keys.**